L-11

Ch-6

ECE 595
Computer Network Systems

Lecture: TCP Congestion Control

Difference betn Reliability & Conjestn Control

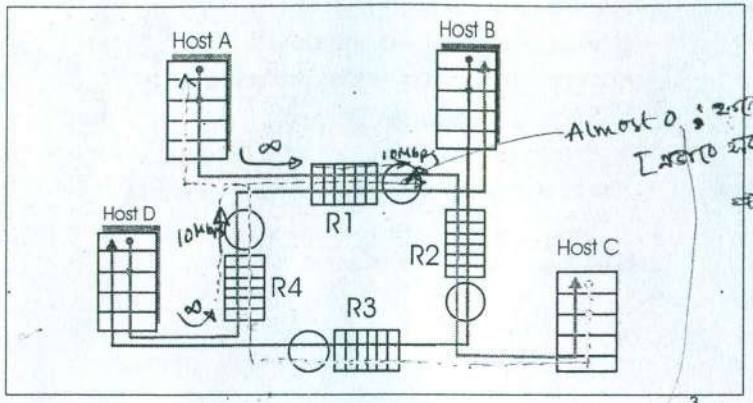
⊛ Reliability ⇒ Rexmit in case of failure
সেন্ড sending rate কমে না

⊛ Conjestn Control ⇒ Sending rate tune করে।

Causes & Costs of Congestion

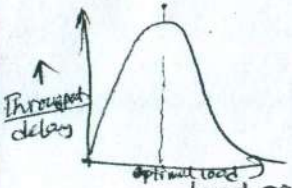
- Four senders – multihop paths
- Timeout/retransmit

Q: What happens as rate increases?

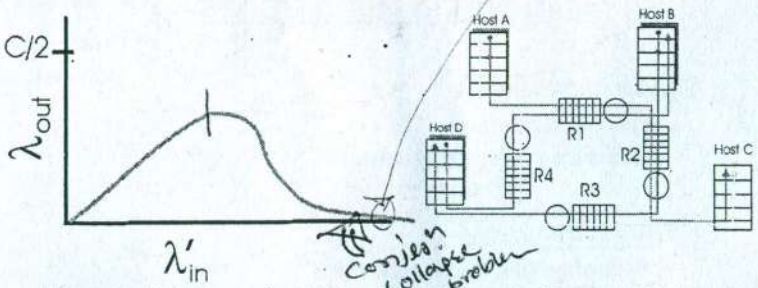


Almost 0 ; R4 এর transmission capacity waste
[অনেক বেশি সার্ভিস দেয়]
⇒ শুধু flow 2 তে router ফিরে এসে
(1st router এর সাথে 2য়ে capacity use
2nd " -1 এর 0 capacity use]

R3 R এর cap 10 Mbps
Host এর rate ∞ এর



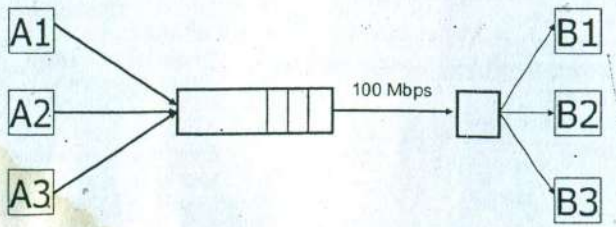
Causes & Costs of Congestion



- When packet dropped, any "upstream transmission capacity used for that packet was wasted!
- Congestion Collapse: *Increase in network load results in decrease of useful work done* network load বাড়ে, useful work কমে

Congestion Control

- What is congestion?
- Why does it occur?



Congestion Control and Avoidance

- A mechanism which:
  - Uses network resources efficiently — efficient use of net resource
  - Preserves fair network resource allocation — fairness
  - Prevents or avoids collapse — avoid collapse
- Congestion collapse is not just a theory
  - Has been frequently observed in many networks

General Approaches (cont'd)

Dynamic Adjustment
- Every sender probe network to test level of congestion
- speed up when no congestion
- slow down when congestion

Approaches
- End system based
  - purely end system based
  - Router hint
- Router based

Disadv of End System based approach
- সবাই slow down হয়ে উচিত হলেও কেউ slow down নাও হতে পারে, এবং malicious হতে পারে, এমনকি unintentional ও হতে পারে। (যেমন: video sharing (সবাইকে slow down হয়ে resolution কমানো উচিত))

Disadv of router based
- NL এর thin property থাকে না

[কারণ, Router এ যদি কাজটা malicious কেউ দিলেই থাকে না] ← contradictory

General Approaches

- Send without care
  - many packet drops
  - could cause congestion collapse
- Reservations
  - pre-arrange bandwidth allocations
  - requires negotiation before sending packets
- Pricing
  - don't drop packets for the high-bidders
  - requires payment model

- Send without care
- Reservation
- Pricing
- Dynamic adjustment

Approaches Towards Congestion Control

- End-end congestion control:
  - No explicit feedback from network
  - Congestion inferred from end-system observed loss, delay
  - Approach taken by TCP
- Network-assisted congestion control:
  - Routers provide feedback to end systems
    - Single bit indicating congestion (SNA, DECbit, TCP/IP ECN, ATM) — congestion indicate by single bit
    - Explicit rate sender should send at — explicit rate for sender
  - Problem: makes routers complicated

NL-এ thin nature থাকছে না

Linear Control

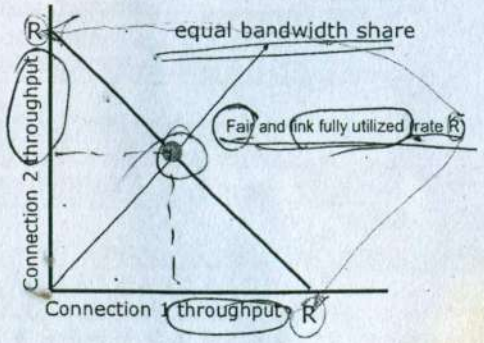
- Many different possibilities for reaction to congestion and probing
  - Examine simple linear controls
    - Window(t + 1) = a + b Window(t)
    - Different a_i/b_i for increase and a_d/b_d for decrease
- Supports various reaction to signals
  - Increase/decrease additively
  - Increased/decrease multiplicatively
  - Which of the four combinations is optimal?

Objectives (in TCP)

- Simple router behavior
- Distributedness
- Efficiency
- Fairness
- Convergence: control system must be stable

Distributed
Efficient
fair
simple
stable

Ideal Operating Point

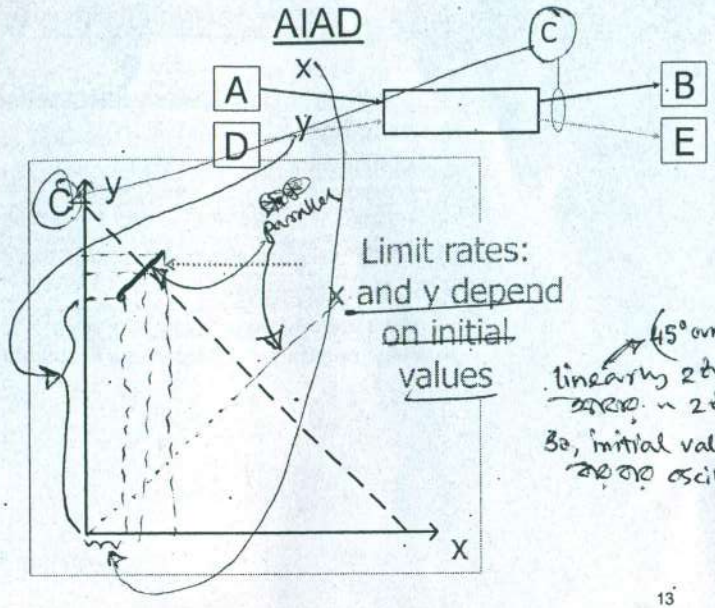


Basic Control Model

- Reduce speed when congestion is perceived
  - How is congestion signaled?
    - Either mark or drop packets
  - How much to reduce?
- Increase speed otherwise
  - Probe for available bandwidth – how?

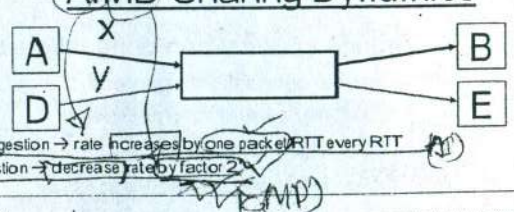
Additive Increase Additive Decrease

AIAD

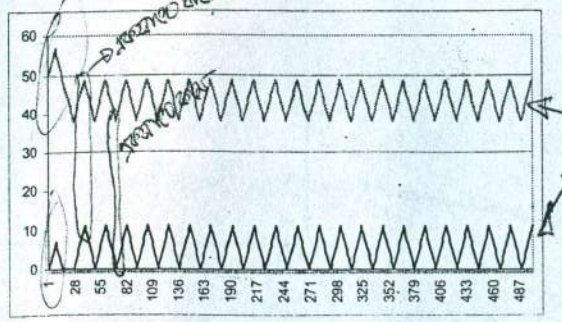


(45° angle এ) linearly 2 থেকে 2 যাচ্ছে না 2 থেকে যাচ্ছে So, initial value থেকে কত কত oscillate.

AIAD Sharing Dynamics



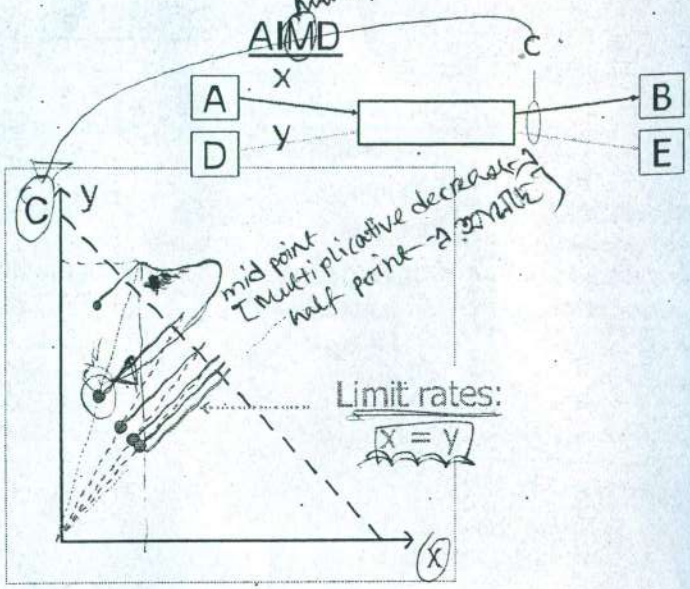
- No congestion → x increases by one packet/RTT every RTT → প্রতি RTT পরপর 1 করে বাড়ছে
- Congestion → decrease x by 1



levels determined by initial values

Multiplicative

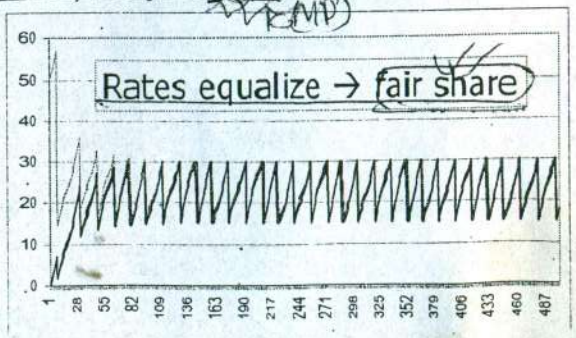
AIMD



mid point [multiplicative decrease → half point → 2 থেকে]

AIMD Sharing Dynamics

- No congestion → rate increases by one packet/RTT every RTT
- Congestion → decrease rate by factor 2



both conn^n এর এরকম হলে, এরকম হবে

BW

Two Basic Components

- Detecting congestion
- Rate adjustment algorithm (change cwnd size)
  - depends on congestion or not

Detect — RTT delay; RTO; Router hint

① RTT becomes high (Harder to track)
② Timeout in sender ⇒ may mislead on lossy medium like wireless!
③ Router can inform sender

⇒ TCP assumes traffic এ loss, wireless-এ loss হলে slow down throughput কমে যাবে।

→ poor performance of TCP on wireless

Detecting Congestion

- Detected based on packet drops
  - Alternative: delay-based methods
- How do you detect packet drops? ACKs
  - TCP uses ACKs to signal receipt of data
  - ACK denotes last contiguous byte received
    - actually, ACKs indicate next segment expected
- Two signs of packet drops
  - No ACK after certain time interval (time-out)
  - Several duplicate ACKs (ignore for now)
- May not work well for wireless networks, why?

① No ack
② Several acks

Packet drop
↳ not good estimate

① pckt drop by { No ack for certain time (Timeout); several acks }
② RTT (Harder)
③ RTO (may mislead in lossy medium)
④ Router info

Sender ⇒ Sender window + Congest. Window + Effective Win
Receiver ⇒ Receiver window + Advertised window (gives max window)

TCP Congestion Control

- TCP connection has window
  - controls number of unacknowledged packets
- Sending rate: Window/RTT — RTT ও window পরিবর্তন
- Vary window size to control sending rate — sender-এ congest. window
- Introduce a new parameter called congestion window (cwnd) at the sender
  - Congestion control is mainly a sender-side operation

Dependence graph

Recvr win → Advertise win; Sender win; Congest. win → Effective win

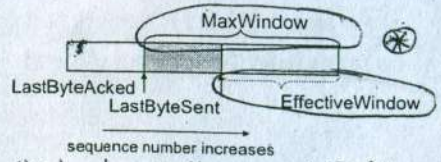
Congestion Window (*cwnd*)

Limits how much data can be in transit — Network এ কতটুকু Data রাখা or limit করা।

change from last time

MaxWindow = min(cwnd, AdvertisedWindow)

EffectiveWindow = MaxWindow – (LastByteSent – LastByteAcked)



advertised win ⇒ flow control (depends on recvr)
congest. win ⇒ congest. control (depends on network)

Adapting cwin

- So far: sliding window + self-clocking of ACKs
- How to know the best cwnd (and best transmission rate)?

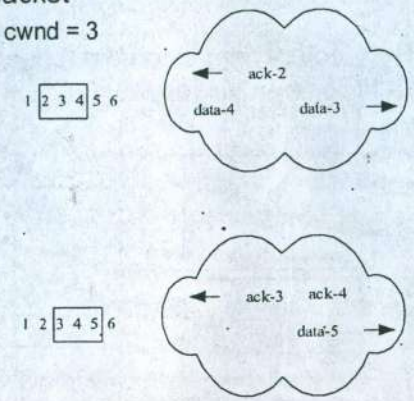
- Phases of TCP congestion control
1. Slow start (getting to equilibrium) — Quickly get a reasonable point
   1. Want to find this very very fast and not waste time
2. Congestion Avoidance > to get steady state
   - Additive increase - gradually probing for additional bandwidth
   - Multiplicative decrease - decreasing cwnd upon loss/timeout

(AIMD)

Slow start
⇓
Congest^n Avoidance

Sliding (Congestion) Window

- Sliding window: each ACK = permission to send a new packet
  - Ex. cwnd = 3



22

Phases of Congestion Control

- **Congestion Window (cwnd)**
  Initial value is 1 MSS (=maximum segment size) counted as bytes
- **Slow-start threshold Value (Congestion Threshold = congthresh)**
  Initial value is the advertised window size

slow start (cwnd < congthresh)
congestion avoidance (cwnd >= congthresh)

Initial
cwnd = 1 MSS
congest^n threshold = Advertise window

Rate Adjustment

- Basic structure:
  - Upon receipt of ACK (of new data): increase rate
    - Data successfully delivered, perhaps can send faster
  - Upon detection of loss: decrease rate

Receive ack: Rate↑ (i.e. successful xmit)
Detect loss: Rate↓ (i.e. unsucc xmit)

TCP: Slow Start

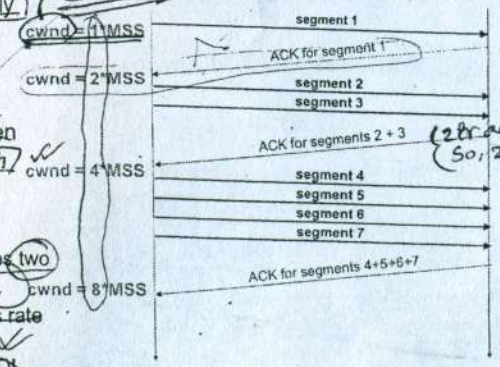
Goal: discover roughly the proper sending rate quickly

Whenever starting traffic on a new connection, or whenever increasing traffic after congestion was experienced:

- Intialize *cwnd* =1 MSS
- Each time a segment is acknowledged, increment *cwnd* by one MSS (*cwnd* += 1 * MSS)
- Continue until
  - Reach congthresh
  - Packet loss

Slow Start Illustration

The congestion window size grows very rapidly



- TCP slows down the increase of *cwnd* when *cwnd >= congthresh*
- Observe:
  - Each ACK generates two packets
  - slow start increases rate exponentially fast (doubled every RTT)

Congestion Avoidance (After Slow Start)

- Slow Start figures out roughly the rate at which the network starts getting congested
- Congestion Avoidance continues to react to network condition
  - Probes for more bandwidth, increase cwnd if more bandwidth available
  - If congestion detected, aggressive cut back cwnd

	Increase in cwnd	
	After each ACK	After each RTT i.e., # of ACKs = cwnd/MSS
Slow start	1 MSS	1 cwnd
Cong^n avoidance	$MSS * \frac{MSS}{cwnd}$	1 MSS

Congestion Avoidance: Additive Increase

- After exiting slow start, slowly increase cwnd to probe for additional available bandwidth
  - Competing flows may end transmission
  - May have been "unlucky" with an early drop
- If (*cwnd > congthresh*) then
  - each time a segment is acknowledged increment *cwnd* by *MSS * (MSS/cwnd)*
- *cwnd* is increased by one MSS only if all segments have been acknowledged
  - Increases by 1MSS per RTT, vs. doubling per RTT

Congestion Avoidance: Multiplicative Decrease

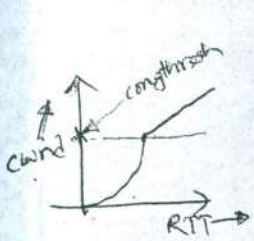
- Timeout = congestion

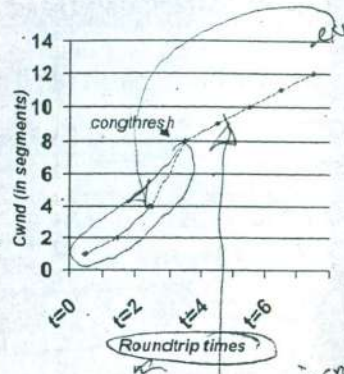
- Each time when congestion occurs,
  - congthresh is set to half the current size of the congestion window:
    - congthresh = cwnd / 2
  - cwnd is reset to one MSS:
    - cwnd = 1 * MSS
  - and slow-start is entered

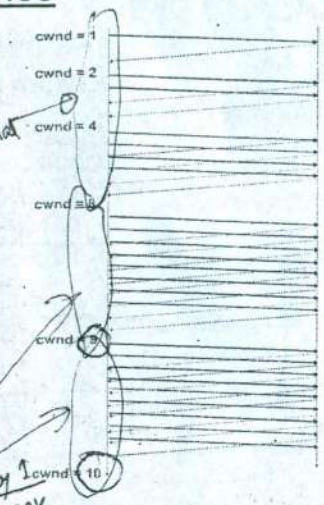
Congestn occurs ⇒
① congthresh = cwnd / 2
② cwnd = 1*MSS
③ Slow start

Example of Slow Start + Congestion Avoidance

Assume that *congthresh = 8*
(cwnd in units of MSS)







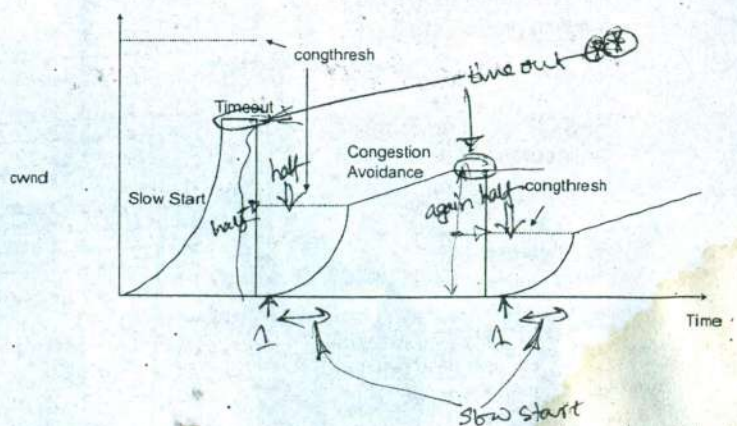
exponential

increase by 1 for each ack [linear]

⇒ Why not start congestn threshold
→ as this threshold is just an initial guess, not actually known
[might be too aggressive]

⊛ Slow start ⇒ cwnd < ssthresh
⊛ congestn avoidance ⇒ cwn ≥ ssthresh

TCP illustration



Why again slow start → resolve in next version of TCP

Detecting Congestion via Timeout

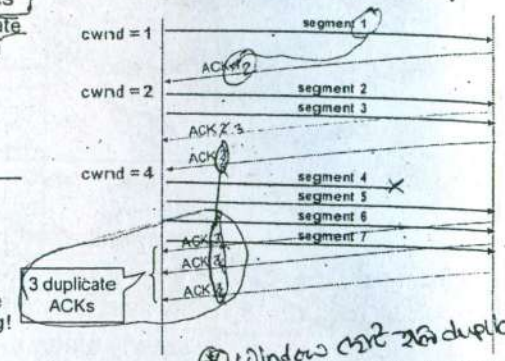
- If there is a packet loss, the ACK for that packet will not be received

- The packet will eventually timeout
  - No ack is seen as a sign of congestion

Fast Retransmit

Duplicate ACK → Out of Sequence segment received

- Resend a segment after 3 duplicate ACKs
  - remember a duplicate ACK means that an out-of-sequence segment was received
  - ACK n means packets 1, ..., n all received



- Notes:
  - duplicate ACKs due to packet reordering!
  - if window is small don't get duplicate ACKs!

ACK 2 (next byte expected from sender) of last byte received

Window এর মধ্যে যদি duplicate ACK আসে না, as window তে থাকা rcvd pckt ignored [কারণ duplicate ACK আসে না]

Responses to Congestion (Loss)

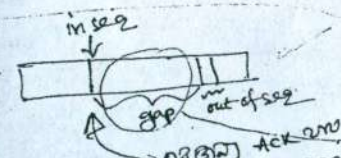
- There are algorithms developed for TCP to respond to congestion
  - TCP Tahoe - the basic algorithm (discussed previously)
  - TCP Reno - Tahoe + fast retransmit & fast recovery
    - Most end hosts today implement TCP Reno

next version

difference in sender side.

- and many more:
  - TCP Vegas (use timing of ACKs to avoid loss)
  - TCP SACK (selective ACK)

[একই same TCP use করে না; ACK, rexmit এগুলো থাকলেও different করে app^n]
Like কেউ slow start, কেউ fast rexmit



দুইবার ACK এলে something is lost.

So, duplicate ACK মানেই something is lost.

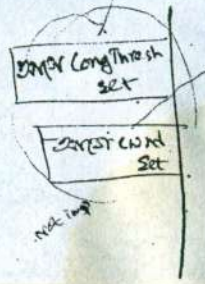
Fast Recovery

- After a fast-retransmit
  - cwnd = cwnd/2 (vs. 1 in Tahoe)
  - congthresh = cwnd
  - i.e. starts congestion avoidance at new cwnd
    - Not slow start from cwnd = 1 * MSS

(fast recovery)

improvement in Reno

- After a timeout
  - congthresh = cwnd/2
  - cwnd = 1 * MSS
  - Do slow start
  - Same as Tahoe



TCP Tahoe

⇒ Timeout
- congthresh = cwnd/2
- cwnd = 1 MSS
- slow start

TCP Reno

⇒ 3 duplicate ACK (Fast Rexmit)
- Rexmit the presumed lost pckt
- cwnd = cwnd/2
- congthresh = cwnd
- congest^n avoidance

fast recovery

⇒ Time out
- Same as Tahoe
- congthresh = cwnd/2
- cwnd = 1 MSS
- slow start

TCP Reno

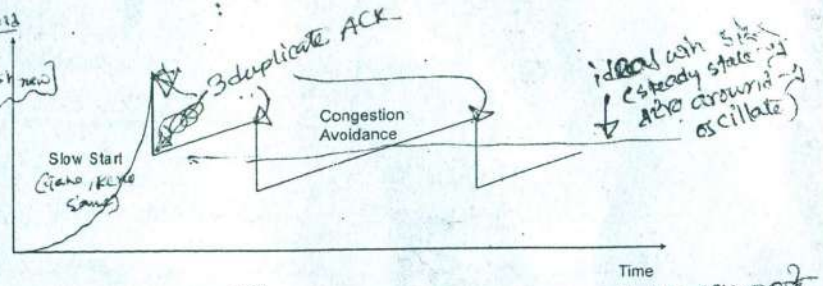
- Problem with Tahoe: If a segment is lost, there is a long wait until timeout
- Reno adds a **fast retransmit** and **fast recovery mechanism**
- Upon receiving 3 duplicate ACKs, retransmit the presumed lost segment ("fast retransmit")
- But do not enter slow-start. Instead enter congestion avoidance ("fast recovery")

⇒ Fast detection of congest^n (before time out)
- change of RTT না করে কোনো প্রকার না করে না;

timeout → } both case congthresh = $\frac{cwn}{2}$
duplicate ack →

difference
timeout: cwnd new = 1 MSS
dup ACK: cwnd new = $\frac{cwnd}{2}$
(i.e. fast recovery) [or, cwnd new = congthresh new]

Fast Retransmit and Fast Recovery



ideal (with size) (steady state) around → oscillate)

- Retransmit after 3 duplicate ACKs
  - prevent expensive timeouts
- Slow start only once per session (if no timeouts)
- In steady state, *cwnd* oscillates around the ideal window size.

only duplicate ack

→ 3 or duplicate ACK পেলে Timeout হবে না wait করে না (starts fast rexmit and fast recovery)

TCP Reno Quick Review

- Slow-Start if cwnd < congthresh
  - cwnd += 1 MSS upon every new ACK (exponential growth)
  - Timeout: congthresh = cwnd/2 and cwnd = 1 MSS
- Congestion avoidance if cwnd >= congthresh
  - Additive Increase Multiplicative Decrease (AIMD)
  - ACK: cwnd = cwnd + MSS * MSS/cwnd
  - Timeout: congthresh = cwnd/2 and cwnd = 1 * MSS

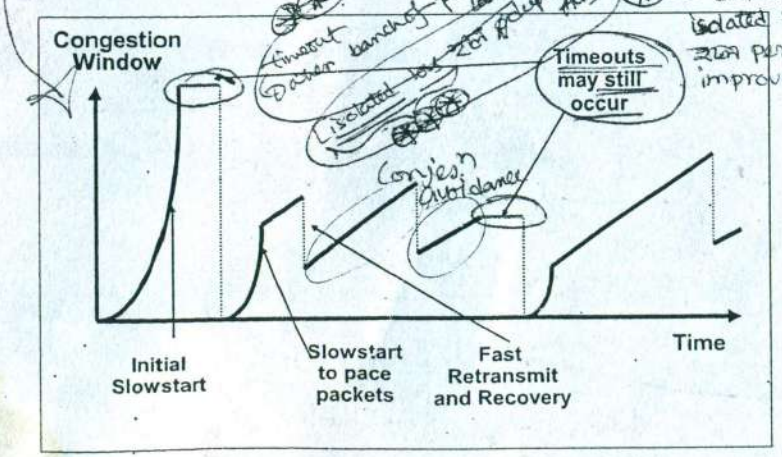
Same as Tahoe

- Fast Retransmit & Recovery
  - 3 duplicate ACKS (interpret as packet loss)
  - Retransmit lost packet (fast Rexmit)
  - cwnd=cwnd/2, congthresh = cwnd (Fast Recovery)
    + Congest. avoidance

Timeout vs duplicate ack
(severally worse than)

Better duplicate ack & timeout

TCP Reno Saw Tooth Behavior



So, Reno isolated pkt loss হলে performance improve হয়।

TCP Congestion Control Summary

- Measure available bandwidth
  - slow start: fast, hard on network
  - AIMD: slow, gentle on network
- Detecting congestion
  - timeout based on RTT
    - robust, causes low throughput
  - Fast Retransmit: avoids timeouts when few packets lost
    - can be fooled, maintains high throughput
- Recovering from loss
  - Fast recovery: don't set cwnd=1 with fast retransmits

Actually হবে cwnd = $\frac{cwnd}{2}$ হবে

Timeout → robust, but low throughput
Fast Rexmit → can be fooled, but high

Timeout → 2*RTT